INTERNAL ENGINEERING DOCUMENTATION

Yuzee Bug Dashboard

Complete Guide & Reference

A single reference for how the dashboard works end to end: the automated bug pipeline it observes (Rollbar, CloudWatch, Jira, Gemini, n8n), every tab in the UI, the Supabase schema behind it, the security model, and a full change log of what has been built and why.

Prepared for: Yuzee Engineering (ashik@yuzee.com)
Covers dashboard state as of: 21 July 2026
Document version: 1.0

Contents

1. Executive Summary

The Yuzee Bug Dashboard is an internal, admin-only Next.js application that gives the engineering team a single place to observe, triage, and act on the automated bug-tracking pipeline. It reads live from a Supabase Postgres database that the n8n automation layer writes to continuously, and it now also hosts a lightweight internal ticketing system for work that isn't an automated bug report.

The dashboard does not replace Rollbar, CloudWatch, Jira, or Gemini — it is the human-facing control panel that sits on top of the data those systems (and the n8n workflows that connect them) produce.

Who it's for

- **Ashik** — admin, oversees the whole pipeline and the engineering team's workload.
- **Junaid** (Backend), **Shaqeeba** (Mobile), **Ramzan** (Web), **Asif** (AI/Data) — the four engineers whose work is routed to them automatically based on where a bug originates.

Tech stack

- Next.js 16 (App Router) + React 19 + TypeScript, deployed with a single admin login (custom cookie session, not Supabase Auth).
- Supabase Postgres for all data, read and written directly from the browser via the anon key, gated by row-level security (RLS) policies rather than Supabase Auth roles.
- Recharts for all analytics charts; lucide-react for icons; no CSS framework — a hand-built dark design system using CSS custom properties.
- Gemini (2.5 Flash / Flash-Lite) for AI triage, summaries, and comment classification, called from n8n and from the dashboard's own AI Analysis panel.

2. Architecture & Data Pipeline

Six n8n workflows populate and maintain the data the dashboard displays. Understanding what each one does makes the dashboard's tabs much easier to read — most tabs are a direct window onto one or two of these workflows.

Workflow	What it does
Bug Automation Rollbar webhook → Jira	Normalizes an incoming Rollbar error, checks suppression rules and duplicates, applies bug_rules overrides, runs Gemini AI triage if no rule fully covers it, decides whether to create/deduplicate/suppress a ticket, writes the enriched row to bug_reports, and creates the Jira ticket (plus a linked YSDT ticket for P1/P2).
Daily Bug Report Cron 9 PM daily	Aggregates the last 24h/7d of bug_reports into KPIs, generates a full HTML report with Gemini, uploads it, and posts a summary card to Microsoft Teams. (The dashboard's Daily Digest tab is ready to display this once a daily_bug_reports table is added to persist it — see §7.)
CloudWatch Bug Poller Cron every 10 minutes	Scans each active CloudWatch log group (tracked in cw_scan_state) for new errors, fingerprints and deduplicates them, and forwards new ones into the same Bug Automation intake as Rollbar errors.
Rule Promoter Cron midnight	Looks for triage_feedback corrections with enough confidence (weight ≥ 3) and automatically promotes them into permanent bug_rules, so the same human correction doesn't have to be made twice.
Feedback Automation Webhook /feedback-intake	Accepts in-app user feedback (not bug reports), classifies it with Gemini, creates a Jira ticket by category (Bug / Feature Request / Support), and stores everything in feedback_reports.
Jira Comment Watcher Cron every 5 minutes	Reads new comments on auto-created Jira tickets, classifies intent with Gemini (mark duplicate / reclassify team / change severity / close / no action), takes the corresponding action on the ticket, and logs every comment it processes to jira_comment_actions.

External systems

- **Rollbar** — automatic error capture for backend (Java/Spring) and web/mobile clients; source of session replay and stack traces.
- **AWS CloudWatch** — backend log groups polled for errors Rollbar doesn't catch directly.
- **Jira (YSC / YSDT)** — ticket of record; YSC holds every bug, YSDT only P1/P2 with an assignee.
- **Gemini** — AI triage, summaries, comment classification, and the dashboard's on-demand AI Analysis panel.
- **PostHog** — session recording integration; wired into the schema but not yet populated by the pipeline (see §7).

3. Tab-by-Tab Walkthrough

The dashboard has ten top-level tabs, in the order they appear in the navigation bar (which scrolls horizontally rather than wrapping, so it never breaks the header layout).

3.1 Overview

The landing page — a single-screen health check of the whole pipeline.

- KPI row: total bugs, P1/critical count, resolved rate, duplicates, Jira coverage, pending review — with a This Week / This Month / Last 3 Months range toggle.
- Daily bug volume chart (7/14/30-day toggle), severity/status/routing/component distribution bars.
- AI Triage Pipeline widget (queued/processed/stale/failed counts, average triage time, stuck-item warning with one-click re-queue).
- Internal Tickets widget — open/in-progress/in-review/done counts with a direct link into the Tickets board.
- Jira Spaces panel, Actionable Insights (auto-generated: Jira-pending alerts, pipeline stalls, component spikes, duplicate rate), and Top Error Clusters.

3.2 Bug Reports

The full, filterable table of every row in bug_reports, with a detailed side panel per bug.

- Filter bar: severity, platform/routing, component, source, status, environment, has-Jira, duplicate, Jira-pending, plus free-text search and a date range.
- 25-row pagination; every column is sortable; each row shows severity, routing, component, description + AI summary, source, device, environment, Jira link, Rollbar link, relative time, and flags.
- Clicking a row opens a side panel with: AI triage detail, the raw Rollbar exception and stack trace, the pre-crash telemetry timeline, active feature flags, debug links (correlation ID → CloudWatch, Rollbar session replay, PostHog), and manual actions (re-queue for AI triage, mark resolved, mark duplicate, create or view a linked Internal Ticket).

3.3 Error Clusters

Bugs grouped by a normalized version of their description, so repeated failures show up as one entry instead of dozens.

- Each cluster shows its dominant severity, routing token, most common component, occurrence count, and first/last seen dates.
- “View N bugs →” jumps to Bug Reports pre-filtered to that exact error pattern; clusters can also be sent straight to the AI Analysis panel.

3.4 Pipeline

System health for the automation itself, not the bugs it produces.

- **Queue Health** — gemini_queue counts (queued/processed/stale/failed) over the last 7 days, average triage time against a 5-minute target, and a re-queue action for anything stuck.
- **Data Quality** — counts of missing correlation IDs, missing Jira keys, incomplete AI triage, and unclassified components — the pipeline's own self-check.
- **CloudWatch** — every polled log group's last-scanned time (flagged if stale ≥ 15 min) and 24h error count (flagged if ≥ 10).

Sub-views: Queue Health · Data Quality · CloudWatch

3.5 Triage & Rules

The governance layer behind the AI triage — what rules exist, what humans have corrected, and what the Jira comment bot has done.

- **Bug Rules** — every regex override that runs before Gemini triage, with priority, forced fields, and an enabled/disabled state.
- **Triage Feedback** — human corrections logged from Jira comments, each with a weight; weight ≥ 3 auto-promotes into a permanent Bug Rule.
- **Jira Comments** — the full log of every comment the automation has classified and acted on, with a 7-day intent-count summary at the top.

Sub-views: Bug Rules · Triage Feedback · Jira Comments

3.6 Feedback

In-app product feedback (a separate stream from automated bug reports).

- Sentiment, category, actionable, severity, and Jira-linkage filters over feedback_reports.
- Currently empty in production (0 submissions to date) — the tab shows an explicit, friendly empty state rather than a blank table.

3.7 Developer

Per-developer workload, derived automatically from routing (not manual assignment).

- Each of the four engineers gets a card: severity breakdown, bugs with no Jira ticket, Jira-pending failures, and — as of this update — an open Internal Tickets count with a link into the board.
- A team summary table ranks developers by P1 load, then total volume.

3.8 Reports

Deeper analytics for retrospectives and reporting upward.

- Six charts: 30-day bug volume by severity, mean time to triage vs. a 5-minute target, resolution rate over 12 weeks, bugs by component, bugs by platform/routing (donut), and P1/P2 response time (creation → Jira ticket) by week.
- An on-demand AI-generated standup summary via Gemini, rate-limited client-side.

3.9 Daily Digest

Placeholder for the nightly HTML report the Daily Bug Report workflow already generates and posts to Teams.

- Shows a clear explanation rather than an error when the daily_bug_reports table doesn't exist yet (it doesn't, today) — see §7 for what's needed to activate it.

3.10 Tickets

A lightweight, internal Jira-alternative for work that isn't an automated bug — refactors, investigations, anything the pipeline wouldn't generate on its own.

- **Board** — a four-column Kanban (To Do / In Progress / In Review / Done); each card shows key, title, priority, type, and assignee, with a one-click “move to next status” action.
- **List** — a sortable/filterable table (status, priority, assignee, type) with the same columns plus created/updated timestamps and a linked-bug column.
- Every ticket has its own detail panel: editable title/description/labels, status/priority/assignee/type dropdowns (each change is written through immediately and recorded in an activity log), a comment thread, and — if it was created from a bug — a live link back to that bug's own detail panel.
- Tickets can be created from scratch (“New Ticket”) or directly from a bug's detail panel (“Create Internal Ticket”, which pre-fills title/description/priority and links the two records together both ways).

Sub-views: Board · List

4. Database Schema Reference

All tables live in the public schema of the spqgjumefasdmgeeokgv Supabase project. This is a summary of purpose and scale, not the full column list (see CLAUDE.md in the repository for that).

Table	Rows	Purpose
`bug_reports`	~330+	The central table. Every automated bug from Rollbar/CloudWatch lands here after full AI triage — severity, ownership, ticketability scoring, device/browser context, Rollbar replay pointers, feature flags at crash time, and the Jira ticket it produced.
`gemini_queue`	~75	Tracks each bug's AI-triage request lifecycle: queued → processed (observed statuses in production today), with timing used for the Pipeline tab's throughput metrics.
`bug_rules`	9	Regex-based overrides applied before Gemini triage — force a severity/category/team/owner, or suppress ticket creation entirely.
`triage_feedback`	5	Human corrections extracted from Jira comments; weight ≥ 3 auto-promotes into bug_rules.
`jira_comment_actions`	100+	Log of every Jira comment the automation has classified and acted on.
`cw_scan_state`	13	Last-scanned bookkeeping per CloudWatch log group, used to detect a stalled poller.
`feedback_reports`	0 (live)	In-app product feedback, separate from bug_reports; AI-classified by category/sentiment.
`bug_suppression_rules`	0 (live)	Pattern-based rules that stop a bug from ever creating a ticket.
`internal_tickets`	new	This session's addition — the Jira-alternative ticket record: key, title, description, type, status, priority, assignee, labels, and an optional link to a bug_reports row.
`internal_ticket_comments`	new	Comment thread per internal ticket.
`internal_ticket_activity`	new	Field-change audit log per internal ticket (status/priority/assignee/type transitions).

Note on internal_tickets.ticket_key — auto-generated as `TIX-1`, `TIX-2`, ... via a Postgres sequence and a BEFORE INSERT trigger, deliberately using a different prefix than Jira's YSC/YSDT so nobody mistakes an internal ticket for a real Jira ticket.

5. Security & Access Model

The dashboard's authentication is a single shared admin login (username/password checked server-side, stored as an HttpOnly cookie) — it is **not** Supabase Auth. Because of that, every read and write the browser makes to Supabase goes through the public anon key, which means row-level security (RLS) policies — not application roles — are the only thing standing between “logged into the dashboard” and “can read/write this table.”

What each table allows

Table(s)	Anon-key access	Why
bug_reports	SELECT	Pre-existing. Dashboard never writes new bug rows directly; it does update status/is_duplicate on existing rows.
gemini_queue	SELECT, INSERT, UPDATE, DELETE (ALL)	Pre-existing, broad by necessity (n8n and the dashboard both write here); flagged by Supabase's own advisor as permissive — not changed by this project.
bug_rules / triage_feedback / jira_comment_actions / cw_scan_state / feedback_reports	SELECT only	Added this project (previously had zero policies — fully unreadable). Read-only was a deliberate, narrower choice than gemini_queue's pattern.
internal_tickets / internal_ticket_comments / internal_ticket_activity	SELECT, INSERT, UPDATE (no DELETE)	Added this session so the Tickets tab can create/edit tickets and comments. No delete policy — closing work out is modeled as a status change (“Done”), not row removal.

Every RLS change in this project has been applied only after explicit, in-the-moment sign-off — read-only grants and read/write grants alike. None grant DELETE, and none were widened beyond what the corresponding UI feature actually needs.

6. Change Log

Session 1 — Pipeline schema catch-up + five new tabs

- Verified the live schema directly rather than trusting either the previous CLAUDE.md or a new prompt file — both were stale in places (e.g. gemini_queue's real status values didn't match what either document claimed).
- Extended the BugReport TypeScript type with ~30 real columns that existed live but weren't yet modeled (device/browser info, Rollbar replay fields, feature flags, ownership fields, ticketability scoring).
- Fixed a real bug: the “Session Replay” quick-link was wired to a PostHog field that's null on every row; added a proper Rollbar replay URL builder and verified it against a bug with real replay data.
- Added read-only RLS policies (with explicit sign-off) so five previously-unreadable tables could be surfaced: bug_rules, triage_feedback, jira_comment_actions, cw_scan_state, feedback_reports.
- Built five new pieces of UI: a Triage & Rules tab (3 sub-views), a CloudWatch pane inside Pipeline, a Feedback tab, and a Daily Digest placeholder tab — plus the nav bar was made horizontally scrollable to fit the growing tab count without breaking the layout.

Session 2 — Internal Tickets (“Jira-lite”) + this guide

- Added a full internal ticketing system: three new tables (internal_tickets, internal_ticket_comments, internal_ticket_activity), a Kanban board, a list view, a detail panel with comments and an activity log, and a creation modal.
- Wired it into the rest of the dashboard rather than leaving it standalone: a bug's detail panel can create or jump to its linked ticket; the Overview tab shows a ticket-status summary; each Developer card shows their open ticket count.
- Added read/write RLS policies (SELECT/INSERT/UPDATE, no DELETE, with explicit sign-off) so the ticket feature can actually persist data through the same anon-key model the rest of the dashboard uses.
- Produced this document.

7. Known Limitations & Out of Scope

- **Daily Digest** is a placeholder until a daily_bug_reports table is added and the Daily Bug Report n8n workflow is updated to write its stats + HTML there instead of only posting to Teams.
- **PostHog session replay** is modeled in the schema (posthog_session_url, posthog_session_id) but not yet populated by the pipeline — the dashboard will pick it up automatically once it is.
- **Internal Tickets** deliberately does not replicate: sprints/epics, story points, file attachments, per-ticket permissions, email notifications, or custom workflows/fields. It covers issue CRUD, a four-status workflow, assignee/priority/type/labels, comments, and a field-change activity log — enough for lightweight internal tracking without becoming a second Jira to maintain.
- **No drag-and-drop** on the ticket board by design — status changes are a single click (“Move to →”), which avoids pulling in a drag-and-drop dependency for a feature this size.

Glossary

Term	Meaning
P1–P4	Severity scale used everywhere in the dashboard (bugs, feedback, and tickets): P1 critical → P4 low.
Routing token	BACKEND / MOBILE / WEB — derived automatically from a bug's platform/category, used to assign it to a developer.
Ticketability score	Gemini's own confidence that a bug is worth creating a Jira ticket for, vs. deduplicating or suppressing it.
TIX-#	Internal ticket key, distinct from Jira's YSC-#/YSDT-# so the two systems are never confused.